

LAB4 Report

LiDar Image Processing

Pin-Jing, Li (111511015 ouo.ee11@nycu.edu.tw)
Ching-Kai, Huang (111511151 kevin.ee11@nycu.edu.tw)
Duan-Kai, Hu (112511015 huduankai@gmail.com)

December 1, 2025

In lab 4 We explored the basic signal processing techniques of a 3D LiDar image.

1 Experiment 1: Distance Detection with Direct Histogram Construction

After pairing each emission with the earliest reception that is not later than it, we compute δt and build histograms over 0-1250 (unit: 100 ps) using 500, 700, 900, 1100, and 1500 bins. The main peak consistently appears near 1000×100 ps, indicating a unimodal distribution with minor noise/multipath influence. With fewer bins (wider bins) the bars look smoother and the peak count appears more concentrated, but the estimate is quantized by a coarser bin width; for 500 bins the δt quantization step is about 2.5×100 ps (distance step $\simeq 3.75$ cm). Increasing to 1500 bins (narrower bins) sharpens the peak and reveals more detail; δt resolvability improves to $\simeq 0.833 \times 100$ ps (distance step $\simeq 1.25$ cm), while per-bin random fluctuation becomes more visible. The main-peak location hardly moves across bin settings, showing that the estimate is insensitive to the choice of binning. Simple conclusion: More bins $\rightarrow$ higher resolution but noisier appearance; Fewer bins $\rightarrow$ smoother plots but larger bin-quantization error. For this dataset, 900-1500 bins provide finer readout without introducing bias and are suitable for the primary distance-estimation display.

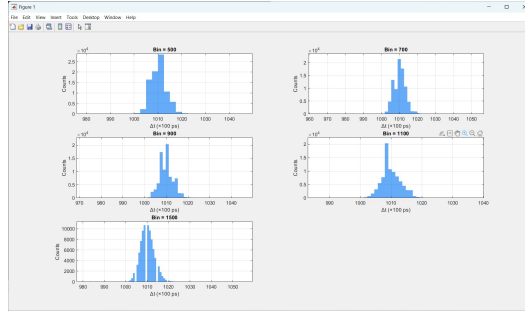


Figure 1: Depth Graph and the detected Edge Graph

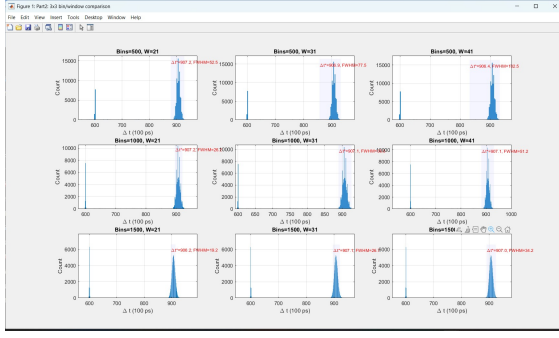
2 Experiment 2: Distance Detection with Window Filter

We first construct ToF samples by keeping, for each emission, only the earliest detected photon and restricting Δt to $[0, 1250]$. On the resulting Δt histogram we evaluate a 3×3 grid of settings: three bin counts (500/1000/1500) and three window widths (21/31/41 bins). We record FWHM, PeakSum, and Density ($= \text{PeakSum}/W$). Across all nine combinations, Δt^* is highly consistent ($\approx 906\text{--}907 \times 100$ ps, distance ≈ 13.6 m), demonstrating that the centroid estimate is robust to binning and window choices.

Differences appear mainly in peak shape and confidence metrics: with a smaller window (21 bins), FWHM is narrowest and Density higher, tightly locking onto the peak core but being more sensitive to occasional spikes; with larger windows (31/41 bins), more tail samples are included, FWHM widens and Density decreases, yielding a smoother peak without shifting its center.

Regarding binning, moving to 1000–1500 bins offers a finer Δt grid; although per-bin counts drop, the sliding window aggregates them (denoising), keeping Δt^* stable and mitigating bin-quantization effects.

Combining the indicators and plots, 1000–1500 bins with a 21–31-bin window is a balanced operating region.



(a) Histograms of the estimated distance with different resolution

Bin	Window	Delta_t_100ps	FWHM_100ps	PeakSum	Density	Distance_m
500	21	907.16	52.5	83350	3969	13.598
500	31	906.92	77.5	83970	2708.7	13.594
500	41	906.39	102.5	84673	2065.2	13.586
1000	21	907.2	26.25	80119	3815.2	13.599
1000	31	907.11	38.75	82882	2673.6	13.597
1000	41	907.08	51.25	83324	2032.3	13.597
1500	21	906.18	19.167	69339	3301.9	13.583
1500	31	907.07	26.667	79596	2567.6	13.597
1500	41	907.02	34.167	82529	2012.9	13.596

(b) Table of the estimated distance with different resolution

Figure 2: Distance Detection with Window Filter

3 Experiment 3: Edge Detection

For this section we perform an edge detection with a linear kernel.

Given a depth map $Z(x, y)$, edges correspond to locations where the surface geometry changes abruptly. These discontinuities manifest as large first- or second-order derivatives. In a discrete image, we approximate derivatives through linear convolution kernels. This section summarizes the mathematical foundations of the filters used in our experiments.

3.1 First- and Second-Order Edge Cues

For a continuous function $Z : \mathbb{R}^2 \rightarrow \mathbb{R}$, the relevant differential operators are:

$$\nabla Z = \left[\frac{\partial Z}{\partial x}, \frac{\partial Z}{\partial y} \right], \quad (1)$$

$$\Delta Z = \frac{\partial^2 Z}{\partial x^2} + \frac{\partial^2 Z}{\partial y^2}. \quad (2)$$

First derivatives (∇Z) measure *changes in slope*, while second derivatives (ΔZ) measure *curvature*. Both are useful for detecting different forms of depth discontinuity.

3.2 Laplacian Filter

The Laplacian operator is a rotation-invariant second derivative:

$$\Delta Z = \frac{\partial^2 Z}{\partial x^2} + \frac{\partial^2 Z}{\partial y^2}. \quad (3)$$

Its discrete approximation (MATLAB `fspecial('laplacian')` with parameter $\alpha = 0$) is:

$$h_{\text{lap}} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (4)$$

The convolution

$$L = Z * h_{\text{lap}} \quad (5)$$

produces large responses where the curvature is high. Laplacian edges are typically thin and sensitive to noise, making the operator effective for highlighting sharp depth transitions.

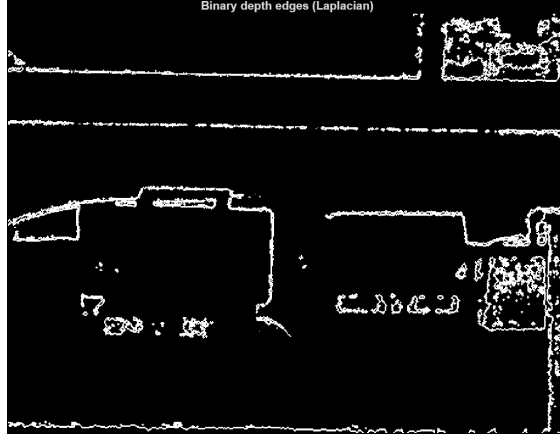


Figure 3: Depth Graph and the detected Edge Graph

3.3 Sobel Operator

The Sobel operator estimates first derivatives with additional smoothing in the direction orthogonal to differentiation. The horizontal and vertical Sobel kernels are:

$$h_x^{\text{sobel}} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, \quad h_y^{\text{sobel}} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}. \quad (6)$$

The gradients are computed as

$$G_x = Z * h_x^{\text{sobel}}, \quad (7)$$

$$G_y = Z * h_y^{\text{sobel}}, \quad (8)$$

and combined into a gradient magnitude:

$$M_{\text{sobel}} = \sqrt{G_x^2 + G_y^2}. \quad (9)$$

Sobel is more robust to noise than pure finite differences due to its built-in smoothing.

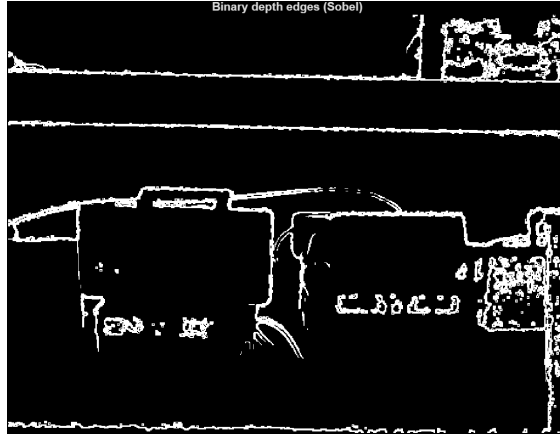


Figure 4: Edge Detection with Sobel Filter

3.4 Prewitt Operator

The Prewitt filter is conceptually similar to Sobel but replaces the Gaussian-like smoothing with a uniform box kernel:

$$h_x^{\text{prewitt}} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, \quad h_y^{\text{prewitt}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}. \quad (10)$$

Its gradient magnitude is defined analogously:

$$M_{\text{prewitt}} = \sqrt{(Z * h_x^{\text{prewitt}})^2 + (Z * h_y^{\text{prewitt}})^2}. \quad (11)$$

Prewitt is computationally simpler but also somewhat more sensitive to noise.

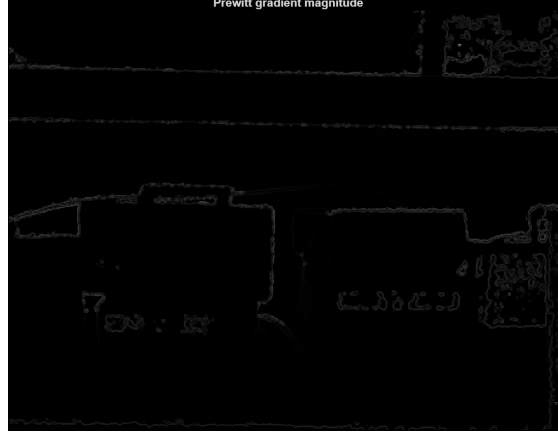


Figure 5: Edge Detection with Prewitt Filter

3.5 Hybrid Laplacian–Sobel Edge Measure

To combine the complementary advantages of first- and second-order cues, we form a convex combination of the Laplacian response and the Sobel gradient magnitude:

$$H = \alpha |Z * h_{\text{lap}}| + (1 - \alpha) M_{\text{sobel}}, \quad 0 \leq \alpha \leq 1. \quad (12)$$

Here,

- $\alpha = 1$ yields a pure Laplacian detector (sharp, thin edges),
- $\alpha = 0$ yields a pure Sobel detector (smooth, stable gradients),
- $0 < \alpha < 1$ provides a multi-scale response capturing both curvature-based and slope-based variations.

This hybrid formulation is particularly effective for depth maps, where both abrupt depth jumps and gradual surface inclinations are informative.

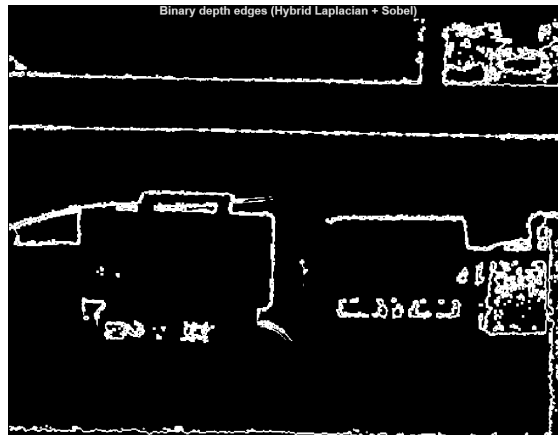


Figure 6: Edge Detection with Interpolation of Two Filters

3.6 Summary

Table 1 summarizes the mathematical structure and behavior of the filters.

Filter	Operator Type	Mathematical Form	Characteristics
Laplacian	2nd derivative	ΔZ	Sharp, thin, noise-sensitive
Sobel	1st derivative + smoothing	$\ \nabla Z\ $	Robust, oriented edges
Prewitt	1st derivative + box smoothing	$\ \nabla Z\ $	Simple, slightly noisier
Hybrid	Combination	$\alpha \Delta Z + (1 - \alpha)\ \nabla Z\ $	Multi-scale, flexible

Table 1: Comparison of edge detection filters for depth images.

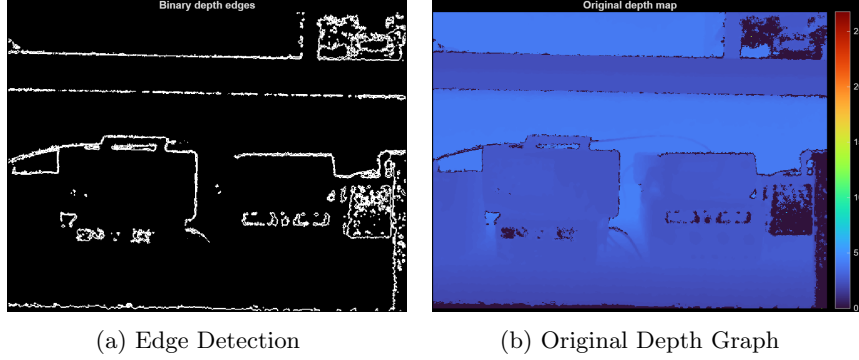


Figure 7: Depth Graph and the detected Edge Graph

4 Experiment 4: Point Cloud Construction, Registration, and Depth Reprojection

In this section we describe the processing pipeline used to convert raw depth images into 3D point clouds, register and merge them, and finally reproject the fused geometry back into a reconstructed depth map. For clarity, the code is divided into several logical stages, each accompanied by a short explanation.

4.1 Loading Depth Metadata and Raw Z16 Images

We first parse camera metadata (intrinsic parameters and resolution) from CSV files and load the corresponding raw depth images stored in 16-bit (Z16) format. The variable `scale` converts depth units (e.g., millimeters to meters), and `flipY` controls vertical image flipping to match the camera coordinate system.

Listing 1: Parsing metadata and loading raw depth maps

```

1 meta1 = parse_depth_meta('part3_full_Depth_metadata.csv');
2 meta2 = parse_depth_meta('part3_full_left_Depth_metadata.csv');
3 meta3 = parse_depth_meta('part3_full_right_Depth_metadata.csv');
4 scale = 1e-3;
5 flipY = true;
6
7 % Load depth images (uint16 Z16 depth)
8 [D1, ~] = readZ16Raw('part3_full_Depth.raw', meta1.W, meta1.H);
9 [D2, ~] = readZ16Raw('part3_full_left_Depth.raw', meta2.W, meta2.H);
10 [D3, ~] = readZ16Raw('part3_full_right_Depth.raw', meta3.W, meta3.H);

```

4.2 Depth-to-Point-Cloud Conversion

Next, each depth map is projected into 3D space using the camera intrinsics contained in the metadata. The function `depthToPC` implements the pinhole projection model to produce MATLAB `pointCloud` objects.

Listing 2: Converting depth images to 3D point clouds

```

1 % Convert depth maps to point clouds
2 pc1 = depthToPC(D1, meta1, scale, flipY);
3 pc2 = depthToPC(D2, meta2, scale, flipY);
4 pc3 = depthToPC(D3, meta3, scale, flipY);

```

4.3 Region-of-Interest Cropping

To focus on the region of interest and discard distant or irrelevant points, we crop the point clouds using an axis-aligned 3D bounding box. The ROI is specified as `[xmin xmax ymin ymax xmin xmax]` in meters.

Listing 3: Cropping point clouds to a 3D ROI

```
1 % ROI cropping: [xmin xmax ymin ymax xmin xmax]
2 roi = [-1 1 -1 1 0.2 3];
3
4 pc1 = select(pc1, findPointsInROI(pc1, roi));
5 pc2 = select(pc2, findPointsInROI(pc2, roi));
6 pc3 = select(pc3, findPointsInROI(pc3, roi));
```

4.4 Noise Filtering and Voxel Downsampling

We then apply statistical denoising to remove isolated outliers, followed by voxel grid downsampling to reduce the point density. The denoising step `pcdenoise` uses a neighborhood-based threshold, while `pcdownsample` averages points inside cubic voxels of a given size.

Listing 4: Denoising and voxel downsampling

```
1 % Remove isolated noise points
2 pc1 = pcdenoise(pc1, 'NumNeighbors', 24, 'Threshold', 1);
3 pc2 = pcdenoise(pc2, 'NumNeighbors', 24, 'Threshold', 1);
4 pc3 = pcdenoise(pc3, 'NumNeighbors', 24, 'Threshold', 1);
5
6 % Downsample using voxel grid (1 cm voxels)
7 voxel = 0.01;
8 f1 = pcdownsample(pc1, 'gridAverage', voxel);
9 f2 = pcdownsample(pc2, 'gridAverage', voxel);
10 f3 = pcdownsample(pc3, 'gridAverage', voxel);
```

4.5 Pairwise ICP Registration

To align the different views into a common coordinate frame, we use Iterative Closest Point (ICP) registration. First, point cloud 2 is registered to point cloud 1 using a point-to-plane metric. We then form an intermediate merged model and register point cloud 3 to this model.

Listing 5: ICP registration between downsampled point clouds

```
1 % Register point cloud 2 to point cloud 1
2 [t21, reg2, rmse21] = pcregistericp(f2, f1, ...
3     'Metric', 'pointToPlane', 'InlierRatio', 0.6, ...
4     'MaxIterations', 100, 'Tolerance', [1e-5 0.5]);
5
6 % Merge pc1 and transformed pc2 into an intermediate model
7 model = pcmerge(pctransform(pc2, t21), pc1, voxel);
8
9 % Register point cloud 3 to the intermediate merged model
10 [t31, reg3, rmse31] = pcregistericp(f3, ...
11     pcdownsample(model, 'gridAverage', voxel), ...
12     'Metric', 'pointToPlane', 'InlierRatio', 0.6, ...
13     'MaxIterations', 100, 'Tolerance', [1e-5 0.5]);
```

4.6 Transforming and Merging Full-Resolution Point Clouds

Once the rigid transforms have been estimated using downsampled data, we apply them to the original (non-downsampled) point clouds. The three aligned point clouds are then merged into a single dense point cloud using a voxel-based fusion.

Listing 6: Applying transforms and merging full-resolution clouds

```
1 % Apply ICP transforms to the original point clouds
```

```

2 pc2_aligned = pctransform(pc2, t21);
3 pc3_aligned = pctransform(pc3, t31);
4
5 % Merge all three point clouds into a unified model
6 merged = pcmerge(pc1, pc2_aligned, voxel);
7 merged = pcmerge(merged, pc3_aligned, voxel);
8
9 % Visualization of pairwise alignment and final merged cloud
10 figure; pcshowpair(pc1, pc2_aligned);
11 title('pc1 (cyan) vs pc2 aligned (magenta)');
12
13 figure; pcshowpair(pc1, pc3_aligned);
14 title('pc1 (cyan) vs pc3 aligned (magenta)');
15
16 figure; pcshow(merged);
17 title('Merged point cloud');
18 xlabel X; ylabel Y; zlabel Z;
19 axis vis3d; view(3);

```

4.7 Reprojection to a Depth Map

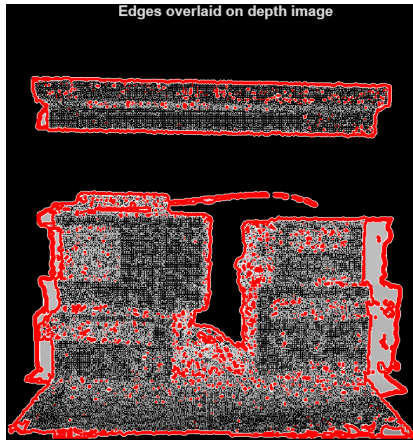
Finally, we project the fused 3D point cloud back into the image plane of the reference camera (here, camera 1) to obtain a reconstructed depth map. The resulting depth image is stored as a raw Z16 file and visualized as a 2D colormap image for inspection.

Listing 7: Reprojecting the merged point cloud to a depth map

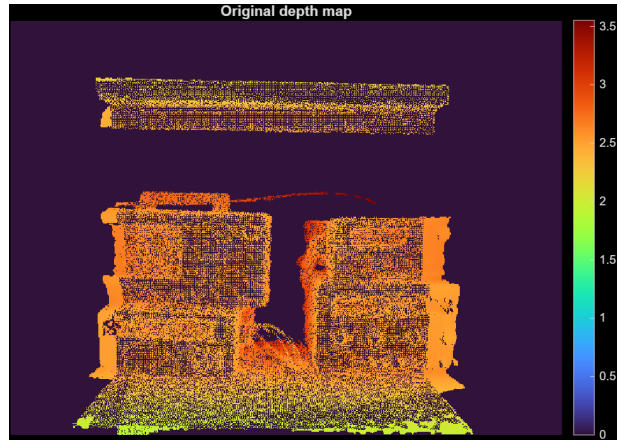
```

1 % Reproject merged point cloud back to the reference camera's depth map
2 D_merged = pcToDepthRaw(merged, meta1, 1e-3, true);
3
4 % Save reconstructed depth as raw uint16
5 fid = fopen('merged_from_pc.raw', 'w');
6 fwrite(fid, D_merged, 'uint16');
7 fclose(fid);
8
9 % Display reconstructed depth map
10 figure;
11 imagesc(double(D_merged) * 1e-3);
12 axis image off;
13 colormap(turbo); colorbar;
14 title('Depth map reprojected from merged point cloud');
15
16 % Save a normalized grayscale visualization to PNG
17 imwrite(mat2gray(double(D_merged)), 'merged_depth_colored.png');
18 fprintf('Saved colored 2D depth map to merged_depth_colored.png\n');

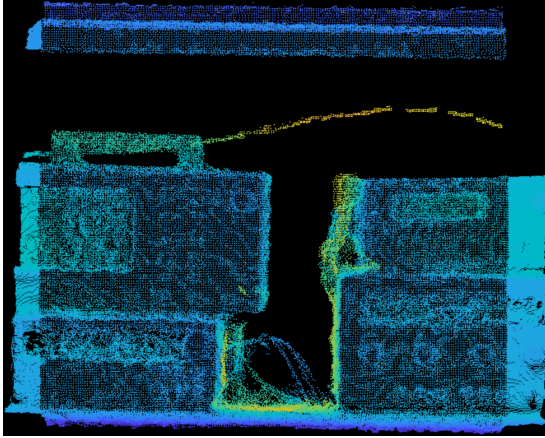
```



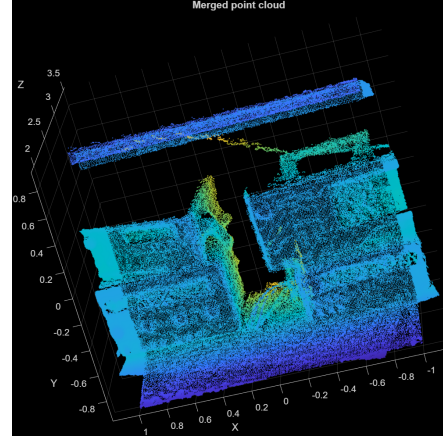
(a) Edge detection of the merged graph



(b) 2D hotmap of the merged graph



(c) merged graph, straight perspective



(d) merged graph, rotated perspective

Figure 8: Depth Graph and the detected Edge Graph

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.